

Investigación TESTING:

i) Planes de Prueba:

- 1) ¿Qué es un plan de pruebas de Software? ¿Cuál es el ciclo de vida de pruebas de Software? Realice un gráfico del ciclo de vida.

Un plan de pruebas es un documento estructurado que indica cómo se llevarán a cabo las pruebas para un proyecto de Software. Describe los objetivos, el alcance, los métodos de prueba, los recursos y el cronograma para la fase de pruebas. En otras palabras es una guía detallada de cómo y qué será evaluado en determinado momento del desarrollo de un proyecto de manera eficiente y efectiva.

El plan de prueba es el encargado de permitir llevar los objetivos de calidad a decisiones prácticas conectando la estrategia de pruebas con la ejecución y de esta forma garantizar que las actividades de prueba sean estructuradas y medibles.

El ciclo de vida de las pruebas de software consta de 6 fases:

1. Análisis de requerimientos:

El equipo QA/testing entiende qué necesita ser testeado.

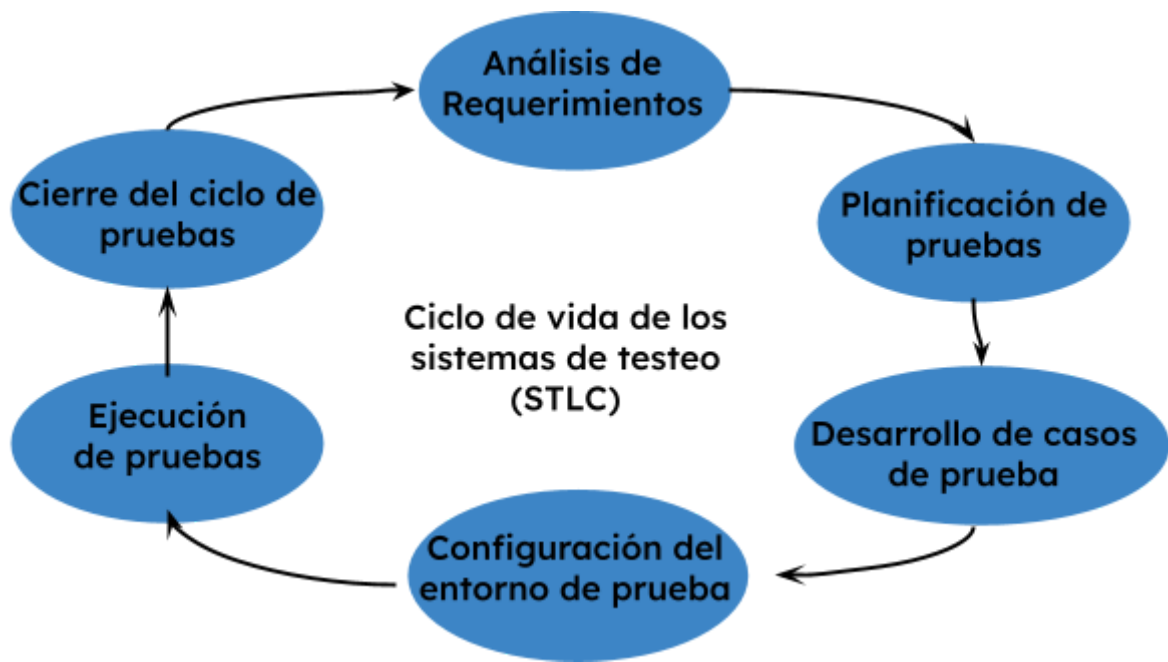
- Se revisa el documento de requerimientos del software y documentación relacionada
- Entrevistan a los clientes para un mejor entendimiento del negocio/detalles adicionales
- Se identifican las ambigüedades, inconsistencias o requerimientos faltantes
- Se revisan los requerimientos no funcionales
- Se identifican riesgos potenciales

2. Planificación de pruebas:

Se define la estrategia y los planes de las pruebas

- Se definen objetivos, alcance y prioridades.
- Se desarrolla una estrategia de prueba que incluye la selección de técnicas de testeo automáticas o manuales.
- Se identifican las herramientas, recursos y el ambiente de testeo requerido.
- Se estiman los costos y recursos necesarios.

- Se incorporan pruebas basadas en riesgo para priorizar las funcionalidades críticas.
3. Desarrollo de casos de prueba:
- Los testers diseñan los casos de prueba de forma detallada
- Identificación de los casos de prueba a desarrollar.
 - Escribir los casos de prueba de forma clara, concisa y fácil de entender.
 - Crear los los datos y escenarios de pruebas que se van a realizar.
 - Identificar los resultados esperados.
 - Revisar y validar los casos de prueba.
4. Configuración de entornos de prueba:
- Se define hardware, software y condiciones de red sobre las cuales serán ejecutados los test
- Configurar el entorno, los browsers, servidores, etc.
 - Preparar credenciales de acceso con sus respectivos permisos.
 - Validar el entorno antes de ejecutar los test.
5. Ejecución de pruebas:
- Se ejecutan los casos de prueba desarrollados en el entorno definido.
- Ejecutar casos de prueba manuales o automatizados.
 - Registrar defectos con detalles como gravedad y prioridad.
 - Recopilar y analizar los resultados de las pruebas.
 - Documentar y compartir los informes de pruebas.
6. Cierre de las pruebas:
- Las actividades de testeo están completas y documentadas
- Se preparan los resúmenes de los resultados obtenidos.
 - Los defectos son seguidos y cerrados.
 - Se limpia el entorno de testeo.
 - Se archivan los casos de testeo, datos y reportes.



ii) Tipos de Prueba:

- 1) Realizar un cuadro con los tipos de prueba, donde se describa cada uno y especifique el objetivo de su ejecución.

Tipo de Prueba	Descripción	Objetivo de Ejecución
Pruebas Unitarias	Consisten en probar por separado la unidad más pequeña de código, como un método o una clase, sin depender de otros componentes.	Detectar errores de lógica en etapas tempranas del desarrollo, cuando su corrección resulta menos costosa.
Pruebas de Integración	Verifican el funcionamiento conjunto de dos o más módulos, como la interacción entre el controlador, el servicio y la base de datos.	Identificar fallas en la comunicación y el intercambio de datos entre módulos, que las pruebas unitarias no logran advertir.

Pruebas del Sistema (End-to-End)	Evalúan la aplicación completa e integrada, reproduciendo recorridos reales del usuario, como registrarse, iniciar sesión, utilizar una funcionalidad y cerrar sesión.	Comprobar que el sistema en su totalidad cumple con los requisitos establecidos.
Pruebas de Aceptación (UAT)	Son realizadas por el cliente o el usuario final, generalmente antes de la puesta en producción, en un entorno similar al real.	Que el cliente valide si el sistema satisface sus necesidades y puede ser entregado.
Pruebas de Regresión	Consisten en volver a ejecutar pruebas ya realizadas luego de una modificación del código, ya sea una corrección o una nueva funcionalidad.	Garantizar que los cambios recientes no hayan afectado funcionalidades que antes operaban correctamente.
Pruebas de Humo (Smoke Testing)	Conjunto reducido de pruebas rápidas sobre las funciones esenciales: que la aplicación inicie, que sea posible ingresar y que las pantallas principales se carguen.	Determinar si la versión es lo suficientemente estable para continuar con pruebas más exhaustivas o si debe ser rechazada.
Pruebas de Rendimiento	Miden el comportamiento del sistema ante distintos volúmenes de usuarios o de datos, considerando los tiempos de respuesta, el consumo de recursos y la estabilidad.	Verificar que el sistema mantiene tiempos de respuesta aceptables y no se degrada cuando aumenta la demanda.
Pruebas de Seguridad	Buscan vulnerabilidades que permitan accesos no autorizados o la exposición de información, como fallas en la autenticación, inyección SQL o control de permisos deficiente.	Proteger los datos de los usuarios y prevenir accesos indebidos al sistema.

iii) Patrones de Automatización de Pruebas.

- 1) ¿Qué son los patrones de automatización de pruebas de software (Modelo de Objeto de Páginas “POM”, Pruebas Basadas en Datos, Desarrollo Impulsado por Comportamiento “BDD”, Pruebas de Carga y Pruebas de Stress)? Realizar una explicación de cada uno de ellos.

Los patrones de automatización de pruebas de software son enfoques estructurados para diseñar, organizar y ejecutar pruebas de forma mantenible, eficiente y exitosa. Dicho de otra manera, los patrones de automatización de software son soluciones reutilizables y probadas para resolver problemas comunes al diseñar y organizar código de pruebas.

- **Modelo de Objeto de Páginas (POM):**
Patrón de diseño de código utilizado generalmente para el desarrollo de interfaces de usuario (UI). Cada página web/componente significativo se representa como una clase en el código, los selectores de los elementos web y los métodos que interactúan con ellos están encapsulados dentro de la clase. Este patrón permite evitar la duplicación de selectores e interacciones en los scripts de prueba. Si, por ejemplo, se modifica la estructura del botón solamente se debe modificar la clase y no todos los archivos de prueba que lo involucren.
- **Pruebas Basadas en Datos:**
Este patrón permite desacoplar la ejecución de los test con los datos de entrada y resultados esperados. El script de prueba contiene la ejecución general mientras que los conjuntos de datos se leen desde fuentes externas. Las pruebas basadas en datos evitan escribir scripts iguales para probar combinaciones distintas de datos.
- **Desarrollo impulsado por Comportamiento (BDD):**
Se define lo que se espera que realice el sistema en lenguaje natural y estructurado. Permite que los requisitos de testeo se interpreten de mejor manera actuando como una especificación funcional, documentación del software y conjunto de pruebas ejecutables. Sintaxis común: Dado que cuando ... entonces ...

- Pruebas de carga:
Prueba de rendimiento que evalúa el comportamiento del sistema bajo niveles de demanda esperados. La idea es simular la simultaneidad de usuarios que se espera que el sistema reciba para medir métricas claves.
- Pruebas de Stress:
Prueba de rendimiento extremo que lleva al sistema a un límite superior al nivel operativo normal. La prueba se trata de progresivamente aumentar la carga o someter a la infraestructura a pruebas extremas hasta provocar un fallo del sistema. Identifica el punto de ruptura (capacidad máxima del software), evalúa cómo se degrada el servicio y observa cómo se recupera una vez que termina la sobrecarga.